

BIPES block_dsl Source Files

Python sources from server/block_dsl.

server/block_dsl/__init__.py

```
"""DSL pipeline for generating Blockly blocks from annotated Python."""

from .pipeline import DefaultPipeline
from .integrate import generate_default_artifacts

__all__ = ["DefaultPipeline", "generate_default_artifacts"]
```

server/block_dsl/builder.py

```
from __future__ import annotations

from .model import (
    BlockKind,
    BlockSpec,
    InputKind,
    InputSpec,
    InstanceMode,
    InstanceReferenceSpec,
    MethodInstanceMode,
    ModuleSource,
)

def snake_case(name: str) -> str:
    out: list[str] = []
    for i, ch in enumerate(name):
        if i > 0 and ch.isupper() and not name[i - 1].isupper():
            out.append("_")
        out.append(ch.lower())
    return "".join(out)

def humanize_identifier(name: str) -> str:
    acronym_words = {
        "adc": "ADC",
        "clk": "CLK",
        "cs": "CS",
        "dat": "DAT",
        "dc": "DC",
        "i2c": "I2C",
        "id": "ID",
        "led": "LED",
        "miso": "MISO",
        "mosi": "MOSI",
        "pwm": "PWM",
        "rgb": "RGB",
        "rst": "RST",
        "rtc": "RTC",
        "scl": "SCL",
        "sck": "SCK",
        "sda": "SDA",
        "spi": "SPI",
        "tx": "TX",
        "rx": "RX",
        "uart": "UART",
    }
    normalized = name.replace("_", " ").replace("-", " ")
    out: list[str] = []
    for i, ch in enumerate(normalized):
        next_ch = normalized[i + 1] if i + 1 < len(normalized) else ""
```



```

        check_type=(
            key_cfg.check_type
            if key_cfg and key_cfg.check_type
            else self._infer_check_type(key_param.type_hint if key_param else None, key_name)
        ),
        default_value=(
            key_cfg.default_value
            if key_cfg and key_cfg.default_value is not None
            else key_param.default_value if key_param else 1
        ),
        is_instance_selector=True,
        options=key_cfg.options if key_cfg else [],
    )
)

for p in fn.parameters:
    cfg = md.param_configs.get(p.name)
    inputs.append(
        InputSpec(
            name=p.name,
            input_kind=cfg.input_kind if cfg and cfg.input_kind else InputKind.INPUT_VALUE,
            check_type=(
                cfg.check_type
                if cfg and cfg.check_type
                else self._infer_check_type(p.type_hint, p.name)
            ),
            default_value=(
                cfg.default_value
                if cfg and cfg.default_value is not None
                else p.default_value
            ),
            options=cfg.options if cfg else [],
        )
    )

inputs = self._order_inputs(inputs)

kind = BlockKind.STATEMENT if fn.is_constructor else (md.kind or BlockKind.STATEMENT)
return BlockSpec(
    type=self._block_type(fn),
    label=self._label_for(fn, inputs),
    category=md.category or "Uncategorized",
    color=md.color,
    tooltip=fn.tooltip,
    help_url=md.url,
    kind=kind,
    is_constructor_block=fn.is_constructor,
    inputs_inline=bool(md.inputs_inline) if md.inputs_inline is not None else False,
    inputs=inputs,
    source_module_name=fn.source_module_name or "library",
    source_function_name=fn.name,
    source_class_name=fn.parent_class_name,
    instance_ref=self._build_instance_ref(fn),
)

def _order_inputs(self, inputs: list[InputSpec]) -> list[InputSpec]:
    id_inputs = [inp for inp in inputs if inp.name == "id"]
    other_inputs = [inp for inp in inputs if inp.name != "id"]
    return id_inputs + other_inputs

def _block_type(self, fn) -> str:
    if fn.parent_class_name:
        suffix = "create" if fn.is_constructor else snake_case(fn.name)
        return f"{snake_case(fn.parent_class_name)}_{suffix}"
    return snake_case(fn.name)

def _label_for(self, fn, inputs: list[InputSpec]) -> str:
    if fn.is_constructor and fn.parent_class_name:
        return f"Create {humanize_identifier(fn.parent_class_name)}"
    return humanize_identifier(fn.name)

def _infer_check_type(self, type_hint: str | None, param_name: str | None = None) -> str | None:

```

```

    if param_name:
        lower_name = param_name.lower()
        if lower_name == "id":
            return "Number"
        if "pin" in lower_name and "pins" not in lower_name:
            return "Number"
    if type_hint in {"int", "float"}:
        return "Number"
    if type_hint == "str":
        return "String"
    if type_hint == "bool":
        return "Boolean"
    return None

def _build_instance_ref(self, fn) -> InstanceReferenceSpec | None:
    md = fn.metadata
    assert md is not None

    if not fn.parent_class_name:
        return None

    instance_mode = md.instance_mode or InstanceMode.SINGLETON
    if instance_mode == InstanceMode.SINGLETON:
        return InstanceReferenceSpec(
            mode=MethodInstanceMode.FIXED_NAME,
            fixed_instance_name=md.instance_name or "board",
        )

    if md.method_instance_mode == MethodInstanceMode.KEY_INPUT:
        return InstanceReferenceSpec(
            mode=MethodInstanceMode.KEY_INPUT,
            key_parameter_name="id",
            key_input_name="id",
        )

    if md.method_instance_mode == MethodInstanceMode.OBJECT_INPUT:
        return InstanceReferenceSpec(mode=MethodInstanceMode.OBJECT_INPUT)

    return None

```

server/block_dsl/emitters.py

```

from __future__ import annotations

import json
import re

from .builder import humanize_identifier, snake_case
from .model import BlockKind, BlockSpec, GeneratorSpec, InputKind, InputSpec
from .model import MethodInstanceMode

_PLACEHOLDER_RE = re.compile(r"\{([A-Za-z_][A-Za-z0-9_]*)\}")

def emit_blockly_blocks_js(blocks: list[BlockSpec]) -> str:
    chunks: list[str] = []
    for block in blocks:
        body = _emit_block_definition(block)
        chunks.append(
            f'Blockly.Blocks["{block.type}"] = {{\n'
            f'  init: function() {{\n{body}\n'
            f'  }}\n'
            f'}};\n'
        )
    return "\n".join(chunks)

def emit_python_generators_js(generators: list[GeneratorSpec], blocks_by_type: dict[str, BlockSpec]) -> str:
    chunks: list[str] = []

```

```

for generator in generators:
    block = blocks_by_type[generator.block_type]
    chunks.append(_emit_python_generator(block, generator))
return "\n\n".join(chunks) + ("\n" if chunks else "")

def emit_definition_markdown(
    category_name: str,
    library_name: str,
    blocks: list[BlockSpec],
    extra_library_names: tuple[str, ...] = (),
) -> str:
    library_names = (library_name, *extra_library_names)
    lines = [
        f"# {category_name}",
        f'<category name="{category_name}">',
        f'<label text="{category_name}"></label>',
    ]
    for name in library_names:
        lines.append(f'<button text="%{{INSTALL_LIBRARY}}: {name}" callbackKey="installPyLib"></button>')
    lines.append("")
    for block in blocks:
        lines.append(f"# {block.type}")
        lines.append(f'<block type="{block.type}">')
        for input_spec in block.inputs:
            shadow = _shadow_for_input(input_spec)
            if shadow:
                lines.extend([
                    f' <value name="{input_spec.name}">',
                    shadow,
                    ' </value>',
                ])
        lines.append("</block>")
        lines.append("")
    return "\n".join(lines).strip() + "\n"

def _emit_block_definition(block: BlockSpec) -> str:
    lines: list[str] = []
    for snippet in _emit_label_inputs(block):
        lines.append(f" {snippet}")

    if block.kind == BlockKind.VALUE:
        lines.append(" this.setOutput(true, null);")
    else:
        lines.append(" this.setPreviousStatement(true, null);")
        lines.append(" this.setNextStatement(true, null);")

    lines.append(f" this.setColour({{block.color if block.color is not None else 230}});")
    lines.append(f" this.setInputsInline({str(block.inputs_inline).lower()});")
    lines.append(f" this.setTooltip({json.dumps(block.tooltip)});")
    lines.append(f" this.setHelpUrl({json.dumps(block.help_url or '')});")
    return "\n".join(lines) + "\n"

def _emit_label_inputs(block: BlockSpec) -> list[str]:
    inputs_by_name = {inp.name: inp for inp in block.inputs}
    used_inputs: set[str] = set()
    tokens: list[tuple[str, str]] = []
    cursor = 0
    for match in _PLACEHOLDER_RE.finditer(block.label):
        if match.start() > cursor:
            tokens.append(("text", block.label[cursor:match.start()]))
            tokens.append(("placeholder", match.group(1)))
            cursor = match.end()
    if cursor < len(block.label):
        tokens.append(("text", block.label[cursor:]))

    emitted: list[str] = []
    pending_text = ""
    for token_type, value in tokens:
        if token_type == "text":

```

```

        pending_text += value
        continue

input_spec = inputs_by_name[value]
used_inputs.add(input_spec.name)
if input_spec.input_kind == InputKind.INPUT_VALUE:
    stmt = f'this.appendValueInput("{input_spec.name}")'
    if input_spec.check_type:
        stmt += f'.setCheck("{input_spec.check_type}")'
    if pending_text.strip():
        stmt += f'.appendField({json.dumps(pending_text.strip())})'
    emitted.append(stmt + ";")
elif input_spec.input_kind == InputKind.FIELD_DROPDOWN:
    options = json.dumps([[label, val] for label, val in input_spec.options])
    label = pending_text.strip() + (" " if pending_text.strip() else "")
    emitted.append(
        'this.appendDummyInput()'
        + (f'.appendField({json.dumps(label)})' if label else "")
        + f'.appendField(new Blockly.FieldDropdown({options}), "{input_spec.name}");'
    )
elif input_spec.input_kind == InputKind.VARIABLE:
    label = pending_text.strip() + (" " if pending_text.strip() else "")
    default_name = input_spec.name
    emitted.append(
        'this.appendDummyInput()'
        + (f'.appendField({json.dumps(label)})' if label else "")
        + f'.appendField(new Blockly.FieldVariable({json.dumps(default_name)}), "{input_spec.name}");'
    )
else:
    label = pending_text.strip() + (" " if pending_text.strip() else "")
    default_value = "" if input_spec.default_value is None else str(input_spec.default_value)
    emitted.append(
        'this.appendDummyInput()'
        + (f'.appendField({json.dumps(label)})' if label else "")
        + f'.appendField(new Blockly.FieldTextInput({json.dumps(default_value)}), "{input_spec.name}");'
    )
pending_text = ""

if pending_text.strip():
    emitted.append(f'this.appendDummyInput().appendField({json.dumps(pending_text.strip())});')

for input_spec in block.inputs:
    if input_spec.name in used_inputs:
        continue
    emitted.append(_emit_unplaced_input(input_spec))

if not emitted:
    emitted.append(f'this.appendDummyInput().appendField({json.dumps(block.label)});')
return emitted

def _emit_unplaced_input(input_spec: InputSpec) -> str:
    field_label = "ID #" if input_spec.name == "id" else humanize_identifier(input_spec.name)
    if input_spec.input_kind == InputKind.INPUT_VALUE:
        stmt = f'this.appendValueInput("{input_spec.name}")'
        if input_spec.check_type:
            stmt += f'.setCheck("{input_spec.check_type}")'
        stmt += f'.appendField({json.dumps(field_label)})'
        return stmt + ";"

    if input_spec.input_kind == InputKind.FIELD_DROPDOWN:
        options = json.dumps([[label, val] for label, val in input_spec.options])
        return (
            'this.appendDummyInput()'
            + f'.appendField({json.dumps(field_label)})'
            + f'.appendField(new Blockly.FieldDropdown({options}), "{input_spec.name}");'
        )

    if input_spec.input_kind == InputKind.VARIABLE:
        default_name = input_spec.name
        return (
            'this.appendDummyInput()'

```

```

        + f'.appendField({json.dumps(field_label)})'
        + f'.appendField(new Blockly.FieldVariable({json.dumps(default_name)}), "{input_spec.name}");'
    )

    default_value = "" if input_spec.default_value is None else str(input_spec.default_value)
    return (
        'this.appendDummyInput()'
        + f'.appendField({json.dumps(field_label)})'
        + f'.appendField(new Blockly.FieldTextInput({json.dumps(default_value)}), "{input_spec.name}");'
    )

def _emit_python_generator(block: BlockSpec, generator: GeneratorSpec) -> str:
    lines = [
        f'Blockly.Python["{block.type}"] = function(block) {{',
        f'  Blockly.Python.definitions_{[json.dumps("import_" + generator.source_module_name)]} = {json.dumps("import " + generator.source_module_name)};',
    ]
    if block.instance_ref and block.instance_ref.mode == MethodInstanceMode.KEY_INPUT:
        registry_name = f'{snake_case(block.source_class_name or "instance")}_instances'
        lines.append(
            f'  Blockly.Python.definitions_{[json.dumps("registry_" + registry_name)]} = {json.dumps(registry_name + " = {}")};'
        )
    for input_spec in block.inputs:
        lines.extend(_emit_python_input_read(input_spec))

    code_expression = _template_to_js_expression(generator.template, block.inputs) + ' + "\\n"'
    lines.append(f"  var code = {code_expression};")

    if block.kind == BlockKind.VALUE:
        lines.append("  return [code.trimEnd(), Blockly.Python.ORDER_NONE];")
    else:
        lines.append("  return code;")
    lines.append("};")
    return "\\n".join(lines)

def _template_to_js_expression(template: str, inputs: list[InputSpec]) -> str:
    names = {inp.name for inp in inputs}
    parts: list[str] = []
    cursor = 0
    for match in _PLACEHOLDER_RE.finditer(template):
        if match.start() > cursor:
            parts.append(json.dumps(template[cursor:match.start()]))
            placeholder = match.group(1)
            parts.append(placeholder if placeholder in names else json.dumps(match.group(0)))
            cursor = match.end()
    if cursor < len(template):
        parts.append(json.dumps(template[cursor:]))
    return " + ".join(parts) if parts else json.dumps(template)

def _emit_python_input_read(input_spec) -> list[str]:
    if input_spec.input_kind == InputKind.INPUT_VALUE:
        return [
            f'  var {input_spec.name} = Blockly.Python.valueToCode(block, {json.dumps(input_spec.name)},'
            f'Blockly.Python.ORDER_ATOMIC);'
        ]
    if input_spec.input_kind == InputKind.FIELD_DROPDOWN:
        value = f'block.getFieldValue({json.dumps(input_spec.name)})'
        if input_spec.check_type == "Number":
            return [f"  var {input_spec.name} = String(Number({value}));"]
        return [f"  var {input_spec.name} = JSON.stringify({value});"]
    if input_spec.input_kind == InputKind.VARIABLE:
        return [
            "  $ = Blockly.internal;",
            f"  var {input_spec.name} ="
            f"${Blockly.Python.namedB_.getName(block.getFieldValue({json.dumps(input_spec.name)})),"
            f"${module$exports$Blockly$Names.NameType.VARIABLE});",
        ]

```

```

raw = f'block.getFieldValue({json.dumps(input_spec.name)})'
if input_spec.check_type == "Number":
    return [f" var {input_spec.name} = String(Number({raw}));"]
return [f" var {input_spec.name} = JSON.stringify({raw});"]

def _shadow_for_input(input_spec) -> str | None:
    if input_spec.input_kind != InputKind.INPUT_VALUE:
        return None

    if _is_pin_sequence_input(input_spec):
        values = input_spec.default_value if isinstance(input_spec.default_value, (list, tuple)) else [None, None, None,
None]
        return _shadow_for_sequence(values, pin_items=True)

    if _is_pin_input(input_spec):
        return _shadow_for_pin(input_spec.default_value)

    if input_spec.check_type == "Boolean" or isinstance(input_spec.default_value, bool):
        field = "TRUE" if bool(input_spec.default_value) else "FALSE"
        return (
            '    <shadow type="logic_boolean">\n'
            f'        <field name="BOOL">{field}</field>\n'
            '    </shadow>'
        )

    if isinstance(input_spec.default_value, (list, tuple)):
        return _shadow_for_sequence(input_spec.default_value)

    if input_spec.check_type == "Number" or isinstance(input_spec.default_value, (int, float)):
        value = 0 if input_spec.default_value is None else input_spec.default_value
        return (
            '    <shadow type="math_number">\n'
            f'        <field name="NUM">{value}</field>\n'
            '    </shadow>'
        )

    text = "" if input_spec.default_value is None else str(input_spec.default_value)
    return (
        '    <shadow type="text">\n'
        f'        <field name="TEXT">{text}</field>\n'
        '    </shadow>'
    )

def _shadow_for_sequence(values, pin_items: bool = False) -> str:
    lines = [
        '    <shadow type="lists_create_with">',
        f'        <mutation items="{len(values)}"></mutation>',
    ]
    for idx, value in enumerate(values):
        lines.append(f'        <value name="ADD{idx}">')
        if pin_items:
            pin_value = "0" if value is None else str(value)
            lines.extend([
                '            <shadow type="pinout">',
                f'                <field name="PIN">{pin_value}</field>',
                '            </shadow>',
            ])
        elif isinstance(value, bool):
            field = "TRUE" if value else "FALSE"
            lines.extend([
                '            <shadow type="logic_boolean">',
                f'                <field name="BOOL">{field}</field>',
                '            </shadow>',
            ])
        elif isinstance(value, (int, float)):
            lines.extend([
                '            <shadow type="math_number">',
                f'                <field name="NUM">{value}</field>',
                '            </shadow>',
            ])
    ]

```

```

        else:
            lines.extend([
                '        <shadow type="text">',
                f'        <field name="TEXT">{value}</field>',
                '        </shadow>',
            ])
            lines.append('        </value>')
        lines.append('    </shadow>')
    return "\n".join(lines)

def _shadow_for_pin(default_value=None) -> str:
    pin_value = "0" if default_value is None else str(default_value)
    return (
        '    <shadow type="pinout">\n'
        f'        <field name="PIN">{pin_value}</field>\n'
        '    </shadow>'
    )

def _is_pin_input(input_spec: InputSpec) -> bool:
    name = input_spec.name.lower()
    pin_aliases = {"sda", "scl"}
    return ("pin" in name and "pins" not in name) or name in pin_aliases

def _is_pin_sequence_input(input_spec: InputSpec) -> bool:
    return "pins" in input_spec.name.lower()

```

server/block_dsl/errors.py

```

class BlockDslError(Exception):
    """Base exception for block DSL failures."""

class AnnotationSyntaxError(BlockDslError):
    """Raised when # @block annotations cannot be parsed."""

class ValidationError(BlockDslError):
    """Raised when resolved metadata is inconsistent."""

class UnknownMetadataKeyError(ValidationError):
    """Raised when metadata contains unsupported keys."""

class LabelPlaceholderError(ValidationError):
    """Raised when a block label references an unknown placeholder."""

class InstanceConfigError(ValidationError):
    """Raised when class instance configuration is incomplete or invalid."""

```

server/block_dsl/extractor.py

```

from __future__ import annotations

import ast
from pathlib import Path
from typing import Any

from .model import (
    ClassBlockSource,
    FunctionBlockSource,
    ModuleSource,
    ParameterSpec,

```

```

    module_name_from_path,
)

def _expr_to_type_name(node: ast.expr | None) -> str | None:
    if node is None:
        return None
    try:
        return ast.unparse(node)
    except Exception:
        return None

def _literal_or_none(node: ast.expr | None) -> Any:
    if node is None:
        return None
    try:
        return ast.literal_eval(node)
    except Exception:
        try:
            return ast.unparse(node)
        except Exception:
            return None

class PythonAstSourceExtractor:
    def extract(self, source: str, path: str | Path = "<memory>") -> ModuleSource:
        tree = ast.parse(source, filename=str(path))
        module_name = module_name_from_path(path)
        module = ModuleSource(name=module_name, path=str(path))

        for node in tree.body:
            if isinstance(node, ast.ClassDef):
                module.classes.append(self._extract_class(node, module_name))
            elif isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                module.functions.append(
                    self._extract_function(node, parent_class_name=None, module_name=module_name)
                )

        return module

    def _extract_class(self, node: ast.ClassDef, module_name: str) -> ClassBlockSource:
        cls = ClassBlockSource(name=node.name, lineno=node.lineno, source_module_name=module_name)
        for child in node.body:
            if isinstance(child, (ast.FunctionDef, ast.AsyncFunctionDef)):
                cls.methods.append(
                    self._extract_function(child, parent_class_name=node.name, module_name=module_name)
                )
        return cls

    def _extract_function(
        self,
        node: ast.FunctionDef | ast.AsyncFunctionDef,
        parent_class_name: str | None,
        module_name: str,
    ) -> FunctionBlockSource:
        fn = FunctionBlockSource(
            name=node.name,
            is_constructor=(node.name == "__init__"),
            parent_class_name=parent_class_name,
            return_type=_expr_to_type_name(node.returns),
            source_module_name=module_name,
            lineno=node.lineno,
        )
        fn.parameters = self._extract_parameters(node, is_method=parent_class_name is not None)
        return fn

    def _extract_parameters(
        self,
        node: ast.FunctionDef | ast.AsyncFunctionDef,
        is_method: bool,
    ) -> list[ParameterSpec]:

```

```

args = list(node.args.args)
defaults = list(node.args.defaults)
default_offset = len(args) - len(defaults)
default_map: dict[str, Any] = {}

for idx, arg in enumerate(args):
    if idx >= default_offset:
        default_map[arg.arg] = _literal_or_none(defaults[idx - default_offset])

params: list[ParameterSpec] = []
for idx, arg in enumerate(args):
    if is_method and idx == 0 and arg.arg in {"self", "cls"}:
        continue
    params.append(
        ParameterSpec(
            name=arg.arg,
            type_hint=_expr_to_type_name(arg.annotation),
            default_value=default_map.get(arg.arg),
        )
    )
return params

```

server/block_dsl/integrate.py

```

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

from .emitters import emit_blockly_blocks_js, emit_definition_markdown, emit_python_generators_js
from .pipeline import DefaultPipeline

@dataclass(slots=True)
class DslTarget:
    source: str
    category_name: str
    library_name: str
    output_block_js: str
    output_generator_js: str
    output_definition_md: str
    extra_library_names: tuple[str, ...] = ()

DEFAULT_TARGETS = [
    DslTarget(
        source="static/page/blocks/libraries/PicoRobotics.py",
        category_name="Robotics Board",
        library_name="PicoRobotics",
        extra_library_names=(),
        output_block_js="static/page/blocks/blocks/roboticsboard_dsl.js",
        output_generator_js="static/page/blocks/pythonic/roboticsboard_dsl.js",
        output_definition_md="templates/page/blocks/definitions/roboticsboard_dsl.md",
    ),
    DslTarget(
        source="static/page/blocks/libraries/sand_table_robot.py",
        category_name="Sand Drawing Machine",
        library_name="sandRobot",
        extra_library_names=("stepper",),
        output_block_js="static/page/blocks/blocks/sand_table_robot_dsl.js",
        output_generator_js="static/page/blocks/pythonic/sand_table_robot_dsl.js",
        output_definition_md="templates/page/blocks/definitions/sand_table_robot_dsl.md",
    ),
    DslTarget(
        source="static/page/blocks/libraries/ds1302.py",
        category_name="DS1302",
        library_name="ds1302",
        output_block_js="static/page/blocks/blocks/ds1302_dsl.js",
        output_generator_js="static/page/blocks/pythonic/ds1302_dsl.js",
    )
]

```

```

        output_definition_md="templates/page/blocks/definitions/ds1302_dsl.md",
    ),
    DslTarget(
        source="static/page/blocks/libraries/dfplayer.py",
        category_name="DFPlayer",
        library_name="dfplayer",
        output_block_js="static/page/blocks/blocks/dfplayer_dsl.js",
        output_generator_js="static/page/blocks/pythonic/dfplayer_dsl.js",
        output_definition_md="templates/page/blocks/definitions/dfplayer_dsl.md",
    ),
    DslTarget(
        source="static/page/blocks/libraries/st7735s.py",
        category_name="ST7735S",
        library_name="st7735s",
        output_block_js="static/page/blocks/blocks/st7735s_dsl.js",
        output_generator_js="static/page/blocks/pythonic/st7735s_dsl.js",
        output_definition_md="templates/page/blocks/definitions/st7735s_dsl.md",
    ),
    # Deprecated in favor of the shared SPI communication blocks.
    # DslTarget(
    #     source="static/page/blocks/libraries/clock_helpers.py",
    #     category_name="Clock Helpers",
    #     library_name="clock_helpers",
    #     output_block_js="static/page/blocks/blocks/clock_helpers_dsl.js",
    #     output_generator_js="static/page/blocks/pythonic/clock_helpers_dsl.js",
    #     output_definition_md="templates/page/blocks/definitions/clock_helpers_dsl.md",
    # ),
]

def generate_default_artifacts(root_path: str | Path) -> None:
    root = Path(root_path)
    pipeline = DefaultPipeline()

    for target in DEFAULT_TARGETS:
        source_path = root / target.source
        if not source_path.exists():
            continue

        source = source_path.read_text(encoding="utf-8")
        result = pipeline.parse(source, source_path)
        blocks = result.blocks
        blocks_by_type = {block.type: block for block in blocks}

        _write_if_changed(
            root / target.output_block_js,
            emit_blockly_blocks_js(blocks),
        )
        _write_if_changed(
            root / target.output_generator_js,
            emit_python_generators_js(result.generators, blocks_by_type),
        )
        _write_if_changed(
            root / target.output_definition_md,
            emit_definition_markdown(
                target.category_name,
                target.library_name,
                blocks,
                extra_library_names=target.extra_library_names,
            ),
        )

def _write_if_changed(path: Path, content: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if path.exists() and path.read_text(encoding="utf-8") == content:
        return
    path.write_text(content, encoding="utf-8")

```

server/block_dsl/metadata.py

```
from __future__ import annotations

from typing import Any

from .errors import UnknownMetadataKeyError
from .model import (
    BlockKind,
    InputKind,
    InstanceMode,
    Metadata,
    MethodInstanceMode,
    ParamConfig,
)

_CLASS_KEYS = {
    "category",
    "color",
    "url",
    "label",
    "kind",
    "inputsInline",
    "instanceMode",
    "instanceName",
    "instanceNameFrom",
    "methodInstanceMode",
}

_FUNCTION_KEYS = {
    "label",
    "category",
    "color",
    "url",
    "kind",
    "inputsInline",
    "params",
}

def metadata_from_dict(raw: dict[str, Any], *, target: str) -> Metadata:
    allowed = _CLASS_KEYS if target == "class" else _FUNCTION_KEYS
    unknown = set(raw) - allowed
    if unknown:
        raise UnknownMetadataKeyError(
            f"Unknown {target} metadata key(s): {'', '.join(sorted(unknown))}"
        )

    md = Metadata(
        category=raw.get("category"),
        color=raw.get("color"),
        url=raw.get("url"),
        label=raw.get("label"),
        kind=BlockKind(raw["kind"]) if raw.get("kind") else None,
        inputs_inline=raw.get("inputsInline"),
        instance_mode=InstanceMode(raw["instanceMode"]) if raw.get("instanceMode") else None,
        instance_name=raw.get("instanceName"),
        instance_name_from=raw.get("instanceNameFrom"),
        method_instance_mode=(
            MethodInstanceMode(raw["methodInstanceMode"])
            if raw.get("methodInstanceMode")
            else None
        ),
    )

    if target == "function":
        params_raw = raw.get("params", {})
        md.param_configs = {
            name: ParamConfig(
                name=name,
                input_kind=InputKind(cfg["inputKind"]) if cfg.get("inputKind") else None,
            )
            for name, cfg in params_raw.items()
        }
```

```

        check_type=cfg.get("checkType"),
        default_value=cfg.get("defaultValue"),
        options=[tuple(opt) for opt in cfg.get("options", [])],
    )
    for name, cfg in params_raw.items()
}

if md.instance_mode == InstanceMode.MULTIPLE:
    # v1+ rule: multiple-instance classes always use a numeric `id`.
    md.instance_name_from = "id"
    if md.method_instance_mode is None:
        md.method_instance_mode = MethodInstanceMode.KEY_INPUT

return md

def merge_metadata(class_md: Metadata | None, method_md: Metadata | None) -> Metadata | None:
    if class_md is None and method_md is None:
        return None
    if class_md is None:
        return method_md
    if method_md is None:
        return class_md

    return Metadata(
        category=method_md.category if method_md.category is not None else class_md.category,
        color=method_md.color if method_md.color is not None else class_md.color,
        url=method_md.url if method_md.url is not None else class_md.url,
        label=method_md.label if method_md.label is not None else class_md.label,
        kind=method_md.kind if method_md.kind is not None else class_md.kind,
        inputs_inline=(
            method_md.inputs_inline
            if method_md.inputs_inline is not None
            else class_md.inputs_inline
        ),
        instance_mode=(
            method_md.instance_mode
            if method_md.instance_mode is not None
            else class_md.instance_mode
        ),
        instance_name=(
            method_md.instance_name
            if method_md.instance_name is not None
            else class_md.instance_name
        ),
        instance_name_from=(
            method_md.instance_name_from
            if method_md.instance_name_from is not None
            else class_md.instance_name_from
        ),
        method_instance_mode=(
            method_md.method_instance_mode
            if method_md.method_instance_mode is not None
            else class_md.method_instance_mode
        ),
        param_configs={**class_md.param_configs, **method_md.param_configs},
    )

```

server/block_dsl/model.py

```

from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum
from pathlib import Path
from typing import Any

class BlockKind(str, Enum):

```

```

STATEMENT = "statement"
VALUE = "value"
HAT = "hat"

class InputKind(str, Enum):
    INPUT_VALUE = "input_value"
    FIELD_DROPDOWN = "field_dropdown"
    FIELD_INPUT = "field_input"
    VARIABLE = "variable"

class InstanceMode(str, Enum):
    SINGLETON = "singleton"
    MULTIPLE = "multiple"

class MethodInstanceMode(str, Enum):
    FIXED_NAME = "fixed_name"
    KEY_INPUT = "key_input"
    OBJECT_INPUT = "object_input"

@dataclass(slots=True)
class ParamConfig:
    name: str
    input_kind: InputKind | None = None
    check_type: str | None = None
    default_value: Any = None
    options: list[tuple[str, str]] = field(default_factory=list)

@dataclass(slots=True)
class Metadata:
    category: str | None = None
    color: int | None = None
    url: str | None = None
    label: str | None = None
    kind: BlockKind | None = None
    inputs_inline: bool | None = None
    instance_mode: InstanceMode | None = None
    instance_name: str | None = None
    instance_name_from: str | None = None
    method_instance_mode: MethodInstanceMode | None = None
    param_configs: dict[str, ParamConfig] = field(default_factory=dict)

@dataclass(slots=True)
class ParameterSpec:
    name: str
    type_hint: str | None = None
    default_value: Any = None

@dataclass(slots=True)
class AnnotatedElement:
    tooltip: str = ""
    is_block_annotated: bool = False
    metadata: Metadata | None = None

@dataclass(slots=True)
class FunctionBlockSource(AnnotatedElement):
    name: str = ""
    is_public_block: bool = False
    is_constructor: bool = False
    return_type: str | None = None
    parameters: list[ParameterSpec] = field(default_factory=list)
    parent_class_name: str | None = None
    source_module_name: str | None = None
    lineno: int | None = None

```

```

@dataclass(slots=True)
class ClassBlockSource(AnnotatedElement):
    name: str = ""
    methods: list[FunctionBlockSource] = field(default_factory=list)
    source_module_name: str | None = None
    lineno: int | None = None

@dataclass(slots=True)
class ModuleSource:
    name: str
    path: str
    classes: list[ClassBlockSource] = field(default_factory=list)
    functions: list[FunctionBlockSource] = field(default_factory=list)

@dataclass(slots=True)
class InputSpec:
    name: str
    input_kind: InputKind = InputKind.INPUT_VALUE
    check_type: str | None = None
    default_value: Any = None
    is_instance_selector: bool = False
    options: list[tuple[str, str]] = field(default_factory=list)

@dataclass(slots=True)
class InstanceReferenceSpec:
    mode: MethodInstanceMode
    fixed_instance_name: str | None = None
    key_parameter_name: str | None = None
    key_input_name: str | None = None

@dataclass(slots=True)
class BlockSpec:
    type: str
    label: str
    category: str
    color: int | None
    tooltip: str
    help_url: str | None
    kind: BlockKind
    is_constructor_block: bool
    inputs_inline: bool
    inputs: list[InputSpec]
    source_module_name: str
    source_function_name: str
    source_class_name: str | None = None
    instance_ref: InstanceReferenceSpec | None = None

@dataclass(slots=True)
class ToolboxCategory:
    name: str
    color: int | None = None
    blocks: list[BlockSpec] = field(default_factory=list)

@dataclass(slots=True)
class GeneratorSpec:
    language: str
    template: str
    block_type: str
    source_module_name: str
    source_function_name: str
    source_class_name: str | None = None

@dataclass(slots=True)
class AnnotationRecord:

```

```

    metadata_dict: dict[str, Any]
    tooltip: str
    annotation_line: int
    target_line: int

@dataclass(slots=True)
class ParseResult:
    module: ModuleSource
    blocks: list[BlockSpec] = field(default_factory=list)
    toolbox: list[ToolboxCategory] = field(default_factory=list)
    generators: list[GeneratorSpec] = field(default_factory=list)

def module_name_from_path(path: str | Path) -> str:
    return Path(path).stem

```

server/block_dsl/pipeline.py

```

from __future__ import annotations

from pathlib import Path

from .builder import BlockModelBuilder
from .extractor import PythonAstSourceExtractor
from .model import ParseResult
from .python_generator import PythonGeneratorBuilder
from .resolver import SourceMetadataResolver
from .scanner import CommentAnnotationScanner
from .toolbox_builder import SimpleToolboxBuilder
from .validation import ModelValidator

class DefaultPipeline:
    def __init__(self) -> None:
        self.scanner = CommentAnnotationScanner()
        self.extractor = PythonAstSourceExtractor()
        self.resolver = SourceMetadataResolver()
        self.validator = ModelValidator()
        self.block_builder = BlockModelBuilder()
        self.toolbox_builder = SimpleToolboxBuilder()
        self.generator_builder = PythonGeneratorBuilder()

    def parse(self, source: str, path: str | Path = "<memory>") -> ParseResult:
        annotations = self.scanner.scan(source)
        module = self.extractor.extract(source, path)
        module = self.resolver.resolve_module(module, annotations)
        self.validator.validate_module(module)
        blocks = self.block_builder.build_blocks(module)
        toolbox = self.toolbox_builder.build_toolbox(blocks)
        generators = self.generator_builder.build_generators(blocks)
        return ParseResult(module=module, blocks=blocks, toolbox=toolbox, generators=generators)

```

server/block_dsl/python_generator.py

```

from __future__ import annotations

from .builder import snake_case
from .model import BlockSpec, GeneratorSpec, MethodInstanceMode

class PythonGeneratorBuilder:
    def build_generators(self, blocks: list[BlockSpec]) -> list[GeneratorSpec]:
        return [
            GeneratorSpec(
                language="python",
                template=self._template_for(block),

```

```

        block_type=block.type,
        source_module_name=block.source_module_name,
        source_function_name=block.source_function_name,
        source_class_name=block.source_class_name,
    )
    for block in blocks
]

def _template_for(self, block: BlockSpec) -> str:
    if block.is_constructor_block:
        return self._constructor_template(block)
    return self._method_template(block)

def _constructor_template(self, block: BlockSpec) -> str:
    cls = block.source_class_name or "UnknownClass"
    args = ", ".join(f"{{{inp.name}}}" for inp in block.inputs if not inp.is_instance_selector)
    class_ref = f"{block.source_module_name}.{cls}"

    if block.instance_ref and block.instance_ref.mode == MethodInstanceMode.FIXED_NAME:
        name = block.instance_ref.fixed_instance_name or "board"
        return f"{name} = {class_ref}({args})"

    if block.instance_ref and block.instance_ref.mode == MethodInstanceMode.KEY_INPUT:
        key = block.instance_ref.key_input_name or "key"
        registry = self._registry_name(block)
        return f"{registry}[{{{key}}}] = {class_ref}({args})"

    return f"{class_ref}({args})"

def _method_template(self, block: BlockSpec) -> str:
    call_args = ", ".join(f"{{{inp.name}}}" for inp in block.inputs if not inp.is_instance_selector)

    if block.instance_ref and block.instance_ref.mode == MethodInstanceMode.FIXED_NAME:
        name = block.instance_ref.fixed_instance_name or "board"
        return f"{name}.{block.source_function_name}({call_args})"

    if block.instance_ref and block.instance_ref.mode == MethodInstanceMode.KEY_INPUT:
        key = block.instance_ref.key_input_name or "key"
        registry = self._registry_name(block)
        return f"{registry}[{{{key}}}].{block.source_function_name}({call_args})"

    return f"{block.source_function_name}({call_args})"

def _registry_name(self, block: BlockSpec) -> str:
    return f"{snake_case(block.source_class_name or 'instance')}_instances"

```

server/block_dsl/resolver.py

```

from __future__ import annotations

from .metadata import merge_metadata, metadata_from_dict
from .model import AnnotationRecord, ModuleSource

class SourceMetadataResolver:
    def resolve_module(
        self,
        module: ModuleSource,
        annotations: dict[int, AnnotationRecord],
    ) -> ModuleSource:
        for cls in module.classes:
            if cls.lineno in annotations:
                record = annotations[cls.lineno]
                cls.is_block_annotated = True
                cls.tooltip = record.tooltip
                cls.metadata = metadata_from_dict(record.metadata_dict, target="class")

        for method in cls.methods:
            if method.lineno in annotations:

```

```

        record = annotations[method.lineno]
        method.is_block_annotated = True
        method.is_public_block = True
        method.tooltip = record.tooltip
        method.metadata = metadata_from_dict(record.metadata_dict, target="function")
    else:
        method.is_public_block = False

    method.metadata = merge_metadata(cls.metadata, method.metadata)

for fn in module.functions:
    if fn.lineno in annotations:
        record = annotations[fn.lineno]
        fn.is_block_annotated = True
        fn.is_public_block = True
        fn.tooltip = record.tooltip
        fn.metadata = metadata_from_dict(record.metadata_dict, target="function")
    else:
        fn.is_public_block = False

return module

```

server/block_dsl/scanner.py

```

from __future__ import annotations

import json
import re

from .errors import AnnotationSyntaxError
from .model import AnnotationRecord

_BLOCK_START_RE = re.compile(r"^\\s*#@block(?:<body>.*)$")
_COMMENT_RE = re.compile(r"^\\s*#(?:<body>.*)$")
_DEF_OR_CLASS_RE = re.compile(r"^\\s*(def|class)\\s+\\w+")

class CommentAnnotationScanner:
    def scan(self, source: str) -> dict[int, AnnotationRecord]:
        lines = source.splitlines()
        records: dict[int, AnnotationRecord] = {}
        i = 0

        while i < len(lines):
            start_match = _BLOCK_START_RE.match(lines[i])
            if not start_match:
                i += 1
                continue

            metadata_dict, next_index = self._read_metadata(lines, i, start_match.group("body"))
            tooltip_lines: list[str] = []
            j = next_index

            while j < len(lines):
                comment_match = _COMMENT_RE.match(lines[j])
                if comment_match and not _BLOCK_START_RE.match(lines[j]):
                    tooltip_lines.append(comment_match.group("body").rstrip())
                    j += 1
                    continue
                break

            while j < len(lines) and lines[j].strip() == "":
                j += 1

            if j >= len(lines) or not _DEF_OR_CLASS_RE.match(lines[j]):
                raise AnnotationSyntaxError(
                    f"@block annotation on line {i + 1} is not followed by a class or def"
                )

```

```

        target_line = j + 1
        records[target_line] = AnnotationRecord(
            metadata_dict=metadata_dict,
            tooltip="\n".join(tooltip_lines).strip(),
            annotation_line=i + 1,
            target_line=target_line,
        )
        i = j + 1

    return records

def _read_metadata(self, lines: list[str], start_index: int, first_body: str) -> tuple[dict, int]:
    json_lines: list[str] = []
    consumed = start_index

    first_body = first_body.strip()
    if first_body:
        json_lines.append(first_body)

    # Support both one-line JSON and multiline commented JSON.
    while True:
        payload = "\n".join(json_lines).strip()
        if payload:
            try:
                return json.loads(payload), consumed + 1
            except json.JSONDecodeError:
                pass

        consumed += 1
        if consumed >= len(lines):
            break

        comment_match = _COMMENT_RE.match(lines[consumed])
        if not comment_match:
            break
        json_lines.append(comment_match.group("body").lstrip())

    raise AnnotationSyntaxError(
        f"Invalid JSON in @block annotation on line {start_index + 1}"
    )

```

server/block_dsl/toolbox_builder.py

```

from __future__ import annotations

from .model import BlockSpec, ToolboxCategory

class SimpleToolboxBuilder:
    def build_toolbox(self, blocks: list[BlockSpec]) -> list[ToolboxCategory]:
        categories: dict[str, ToolboxCategory] = {}

        for block in blocks:
            category = categories.get(block.category)
            if category is None:
                category = ToolboxCategory(name=block.category, color=block.color)
                categories[block.category] = category
            category.blocks.append(block)

        return list(categories.values())

```

server/block_dsl/validation.py

```

from __future__ import annotations

import re

```

```

from .errors import InstanceConfigError, LabelPlaceholderError, ValidationError
from .model import InstanceMode, InputKind, MethodInstanceMode, ModuleSource

_PLACEHOLDER_RE = re.compile(r"\{([A-Za-z_][A-Za-z0-9_]*)\}")

class ModelValidator:
    def validate_module(self, module: ModuleSource) -> None:
        for cls in module.classes:
            self._validate_class(cls)
            for method in cls.methods:
                self._validate_function(method)

        for fn in module.functions:
            self._validate_function(fn)

    def _validate_class(self, cls) -> None:
        md = cls.metadata
        if not md:
            return

        instance_mode = md.instance_mode or InstanceMode.SINGLETON
        if instance_mode == InstanceMode.MULTIPLE:
            constructor = next((method for method in cls.methods if method.is_constructor), None)
            if constructor is None:
                raise InstanceConfigError(
                    f"Class {cls.name}: instanceMode=multiple requires an annotated __init__ constructor"
                )

            constructor_param_names = {p.name for p in constructor.parameters}
            if "id" not in constructor_param_names:
                raise InstanceConfigError(
                    f"Class {cls.name}: instanceMode=multiple requires constructor parameter 'id'"
                )

    def _validate_function(self, fn) -> None:
        if not fn.is_public_block:
            return

        md = fn.metadata
        if md is None:
            raise ValidationError(f"{fn.name}: public block is missing resolved metadata")

        param_names = {p.name for p in fn.parameters}
        for cfg_name, cfg in md.param_configs.items():
            if cfg_name not in param_names:
                raise ValidationError(
                    f"{fn.name}: param config '{cfg_name}' does not match a function parameter"
                )
            if cfg.input_kind == InputKind.FIELD_DROPDOWN and not cfg.options:
                raise ValidationError(
                    f"{fn.name}: field_dropdown param '{cfg_name}' requires options"
                )
            if cfg.options and cfg.input_kind not in {InputKind.FIELD_DROPDOWN, None}:
                raise ValidationError(
                    f"{fn.name}: options only make sense for field_dropdown params"
                )

        if md.label:
            valid_names = set(param_names)
            if (
                fn.parent_class_name
                and (md.instance_mode or InstanceMode.SINGLETON) == InstanceMode.MULTIPLE
                and md.method_instance_mode == MethodInstanceMode.KEY_INPUT
                and not fn.is_constructor
            ):
                valid_names.add("id")

            for placeholder in _PLACEHOLDER_RE.findall(md.label):
                if placeholder not in valid_names:
                    raise LabelPlaceholderError(

```

```
f"{fn.name}: label placeholder '{placeholder}' is not a valid input name"  
)
```